

DAGFormer: Learned Computation Graphs for Transformers

Progress Report — March 2026

This document summarizes our experiments on learning input-dependent computation graphs (DAGs) for transformer language models. We cover two main lines of work: **midtraining** on a pretrained OLMo-1B, and **pretraining from scratch** with a 300M-parameter model. We present results, analyze failure modes, and outline promising next steps.

Key finding: Midtraining on pretrained models failed because $A=1$ (dense) is a flat optimum in the loss landscape. Pretraining from scratch shows that self-embed predictors *can* learn meaningful sparse topologies, but a 3.5x throughput overhead makes direct comparison with dense baselines unfair. We are currently testing staged, alternating, and improved predictor designs to resolve this tension.

1. Introduction

Standard transformers use a fixed sequential computation graph: layer 0 $\rightarrow$ layer 1 $\rightarrow$... $\rightarrow$ layer L . Every input sees the same wiring regardless of content. **DAGFormer** replaces this rigid structure with a learned, input-dependent directed acyclic graph (DAG) that controls which attention heads feed into which other heads across layers.

The core modification: each attention head j in layer l receives a **gated combination** of prior heads' outputs, controlled by an adjacency matrix $A \in [0,1]^{N \times N}$ ($N = \text{layers} \times \text{heads}$). When $A[i,j] = 1$ for all valid cross-layer pairs, the behavior is identical to the standard residual stream (baseline reproduction). The matrix A is produced by a **structure predictor** that conditions on the input context.

1.1 Oracle Search Motivation

An oracle search (500 gradient steps per 1024-token window, directly optimizing A) showed that context-dependent topologies can reduce NLL from 2.58 to 0.12 on OLMo-1B (median across 50 windows, 100% improved). This demonstrates enormous headroom, but the per-window optimization is far too slow for practical use. The goal of DAGFormer is to **amortize** this search into a learned predictor.

1.2 Architecture Overview

The pipeline has two components: (1) a **structure predictor** that maps the input context to an adjacency matrix A via a low-rank parameterization ($Z = UV^T + \text{bias} \rightarrow \text{Gumbel-Sigmoid} \rightarrow \text{cascading gate}$), and (2) a **modified transformer forward pass** that assembles per-head inputs according to A . Training is end-to-end: NLL gradients flow through the modified forward back to the predictor.

See `structure_predictor.html` for the full interactive architecture diagram.

2. Midtraining on Pretrained OLMo-1B

Our first approach was to keep the pretrained OLMo-1B and learn topology on top of it. We explored two phases: **Phase 1** (frozen OLMo, only predictor trains) and **Phase 2** (unfrozen OLMo, joint training). Both ultimately failed.

2.1 Phase 1: Frozen OLMo — A=1 Is a Flat Optimum

With OLMo frozen, any deviation from dense connectivity (A=1) hurts performance. We tested multiple `init_logit` values that control how far from dense the predictor starts:

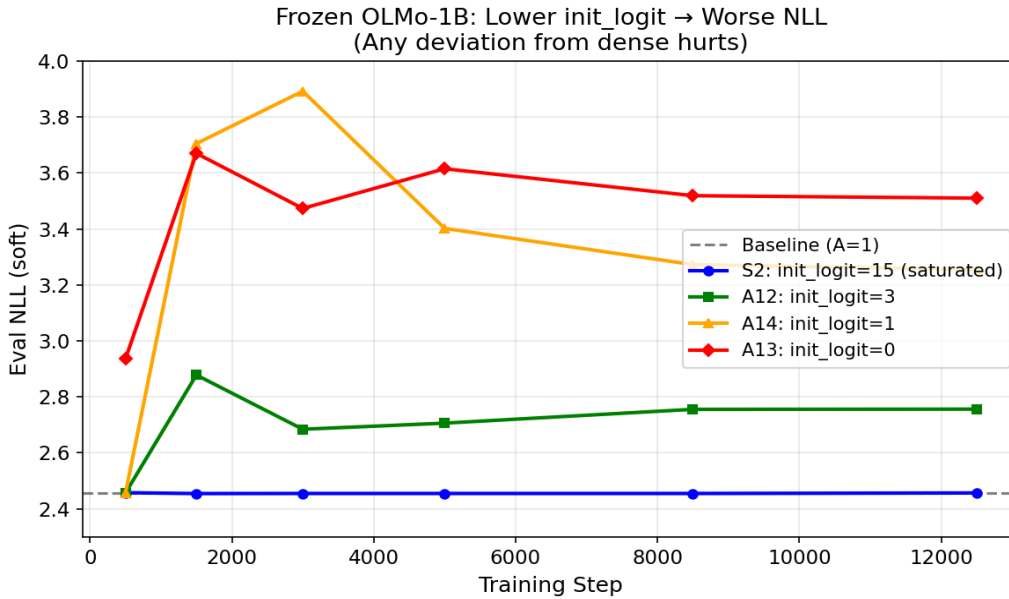


Figure 1. Frozen OLMo-1B: lower `init_logit` → further from dense → worse NLL. S2 (`init_logit`=15) stays at baseline but sigmoid is saturated (no gradient). A12/A13/A14 have gradient flow but converge to fixed topologies worse than dense.

To understand *why* A=1 is optimal, we computed $\partial \text{Loss} / \partial A$ at A=1 across 50 eval windows:

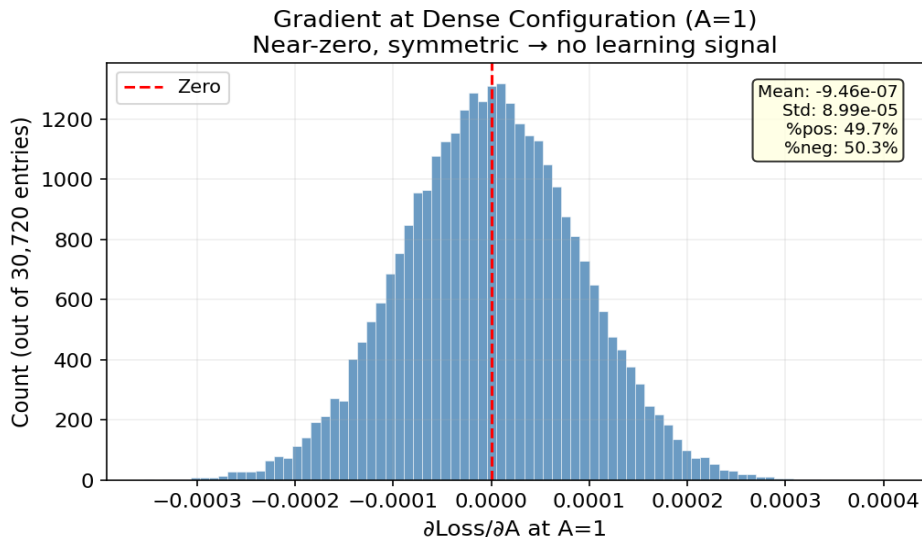


Figure 2. Gradient at $A=1$ is symmetric around zero (49.7% positive, 50.3% negative) with magnitude $\sim 4 \times 10^{-5}$. There is no consistent direction to move. The pretrained weights have made $A=1$ a flat plateau in the loss landscape.

2.2 Phase 2: Unfrozen OLMo — The LR Dilemma

Unfreezing OLMo allows the model to adapt to non-dense topologies. We ran 10 experiments varying OLMo LR, predictor LR, temperature schedule, initialization, and sparsity pressure.

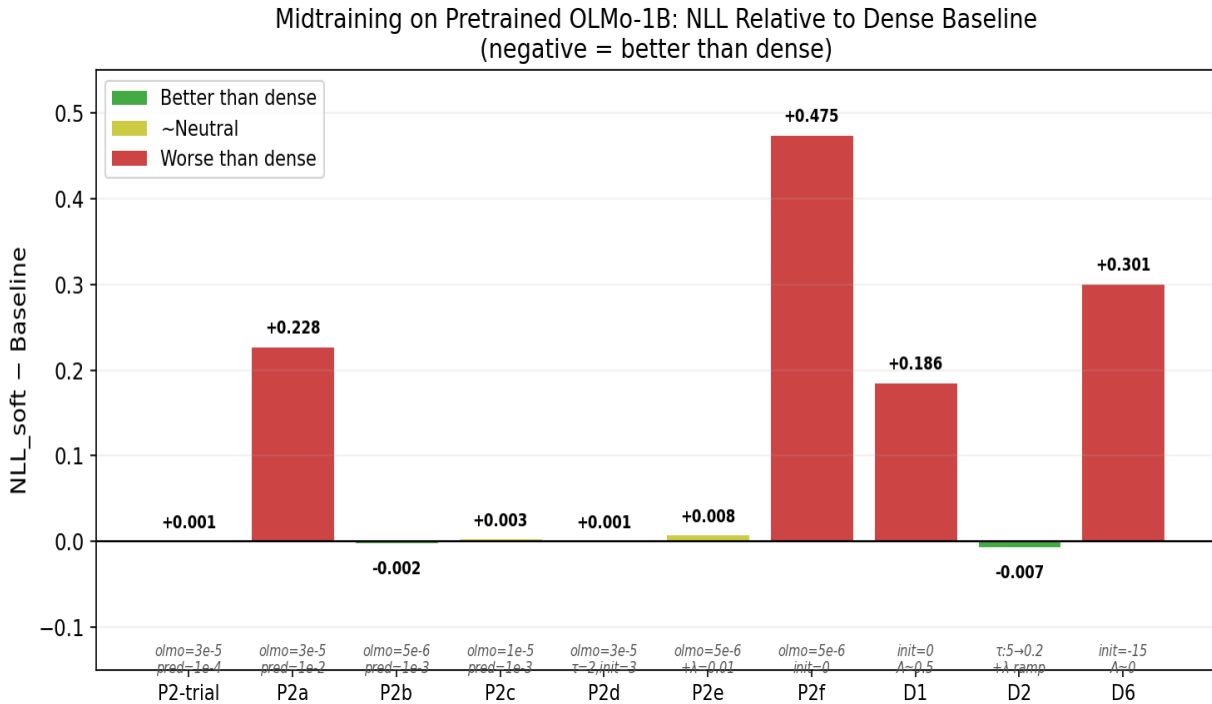


Figure 3. NLL relative to dense baseline for all midtraining experiments. Only P2b (-0.002) and D2 (-0.007) show any improvement, and both remain near-dense (mean_A > 0.97). Most experiments are substantially worse.

The core dilemma:

- **High OLMo LR:** OLMo quickly adapts to dense via continual pretraining. The predictor has no incentive to deviate from $A \approx 1$.
- **Low OLMo LR:** OLMo cannot adapt to non-dense topologies. Any deviation from $A=1$ destroys information the model expects.
- **High predictor LR:** Topology collapses to random (P2a: mean_A=0.5, NLL +0.23).
- **Starting sparse** (D1, D6): OLMo is damaged by missing connections it relies on.

Best midtraining result: D2

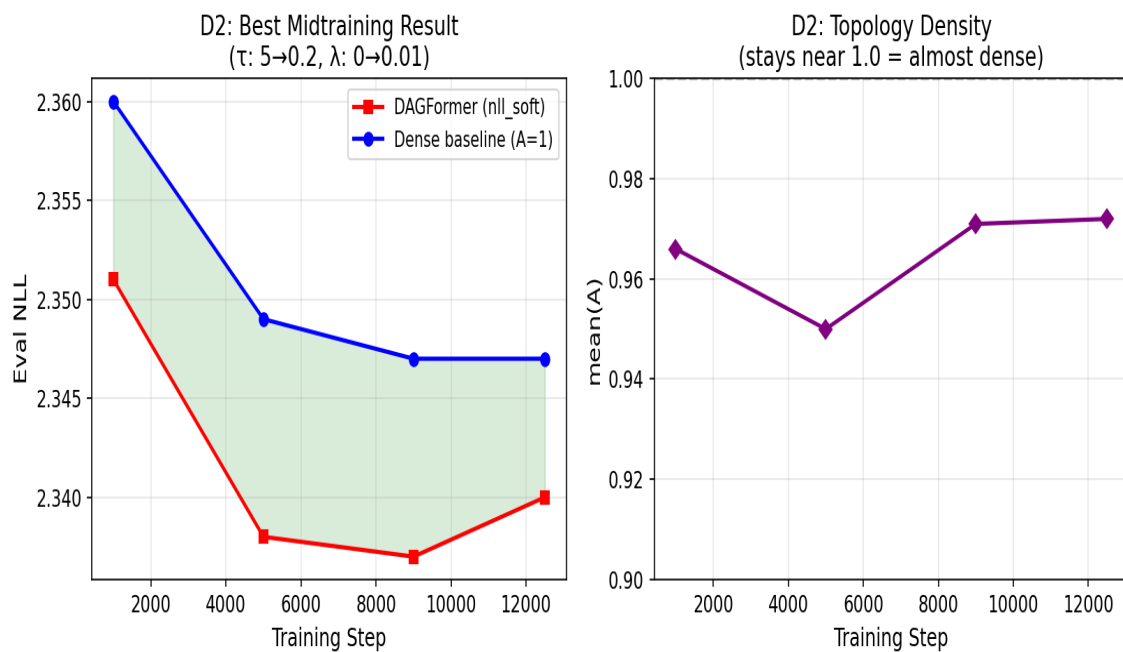


Figure 4. D2 (τ : 5 $\rightarrow$ 0.2 + sparsity ramp) achieved -0.007 NLL vs baseline, but topology stayed near-dense (mean_A $\approx$ 0.97) and jaccard_var = 0 throughout (no context-dependent routing). The improvement is real ($p < 10^{-6}$, $n=500$ paired t-test) but tiny and context-independent.

2.3 Summary: All Midtraining Experiments

Exp	OLMo LR	Pred LR	Strategy	NLL-Base	mean_A	Jaccard
P2-trial	3e-5	1e-4	Default	+0.001	0.98	0
P2a	3e-5	1e-2	High pred LR	+0.228	0.50	0
P2b	5e-6	1e-3	Low OLMo LR	-0.002	0.99	0
P2c	1e-5	1e-3	Med OLMo LR	+0.003	0.99	0
P2d	3e-5	1e-3	Low tau+init	+0.001	0.99	0
P2e	5e-6	1e-3	+Sparsity	+0.008	0.97	0
P2f	5e-6	1e-3	init_logit=0	+0.475	0.56	0
D1	5e-6	1e-3	A starts ~0.5	+0.186	0.71	0
D2	5e-6	1e-3	tau anneal+lam	-0.007	0.97	0
D6	5e-6	1e-3	A starts ~0	+0.301	0.56	0

Table 1. All midtraining experiments on OLMo-1B. jaccard_var=0 means the predictor produces identical topology for all inputs (no context-dependence). NLL-Base is eval/nll_soft minus the dense baseline at the same step.

2.4 Conclusion: Why Midtraining Failed

Pretrained transformer weights are optimized for dense ($A=1$) connectivity. The loss landscape at $A=1$ is a **flat plateau** with near-zero gradient, making gradient-based topology learning impossible without fundamentally restructuring the model. Joint training (Phase 2) creates a chicken-and-egg problem: the model adapts to dense faster than the predictor can propose useful alternatives. **We need to train from scratch** so that weights and topology co-evolve from random initialization.

3. Pretraining from Scratch — 300M OLMo-2

Given the failure of midtraining, we switched to pretraining a smaller model (300M parameters, OLMo-2 architecture: 12 layers $\times$ 16 heads) from random initialization with DAGFormer topology. The A matrix is 192 $\times$ 192. We first trained a **dense baseline** (no topology) for comparison, then trained DAGFormer variants with the same token budget (5.24B tokens).

3.1 Dense Baseline

Standard OLMo-2 300M trained for 10,000 steps (5.24B tokens) on Dolma v1.7. Throughput: 81K tokens/sec on 4 $\times$ A40. Final eval NLL: **3.6094**. This is the target to beat.

3.2 DAGFormer Variants (Batch 1: From Scratch)

We tested three predictor types, all training from random init with the same 5.24B token budget:

Variant	Predictor	tok/s	Steps done	Eval NLL (soft)	Own Baseline	mean_A
Baseline	None (dense)	81K	10000	3.609	-	1.0
Static	Learnable UV^T (no input dep.)	23K	~4690	~5.89	~5.98	0.98
Self-embed	Mean-pool embeddings	23K	~4670	~5.75	~5.96	0.42
Qwen	Frozen Qwen-0.6B (external)	14K	~2920	~6.15	~6.28	0.41

Table 2. From-scratch pretraining results. All DAGFormer variants are far behind the baseline in absolute NLL because they see ~3.5x fewer tokens in the same wall time.

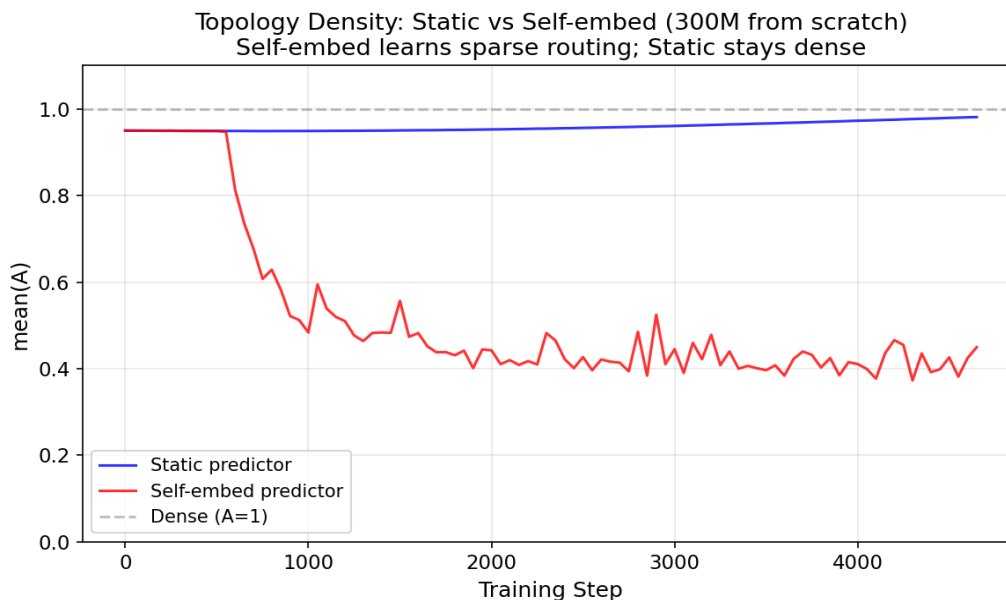


Figure 5. Topology density over training. Self-embed rapidly learns sparse routing (mean_A $\rightarrow$ 0.42), while Static stays dense. This is the **first successful topology learning** across all experiments.

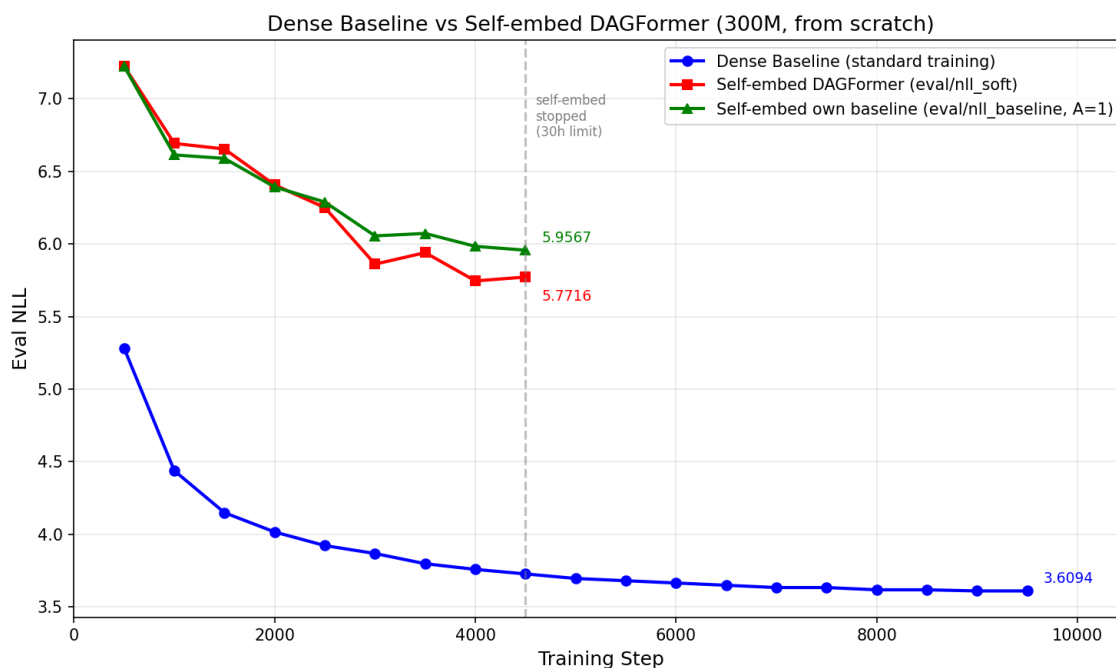


Figure 6. Eval NLL comparison. Blue: dense baseline (standard training). Red: self-embed DAGFormer. Green: self-embed's own dense forward ($A=1$). Key insight: red < green after step 3000, proving learned topology helps the same undertrained model. But both are far behind the blue baseline.

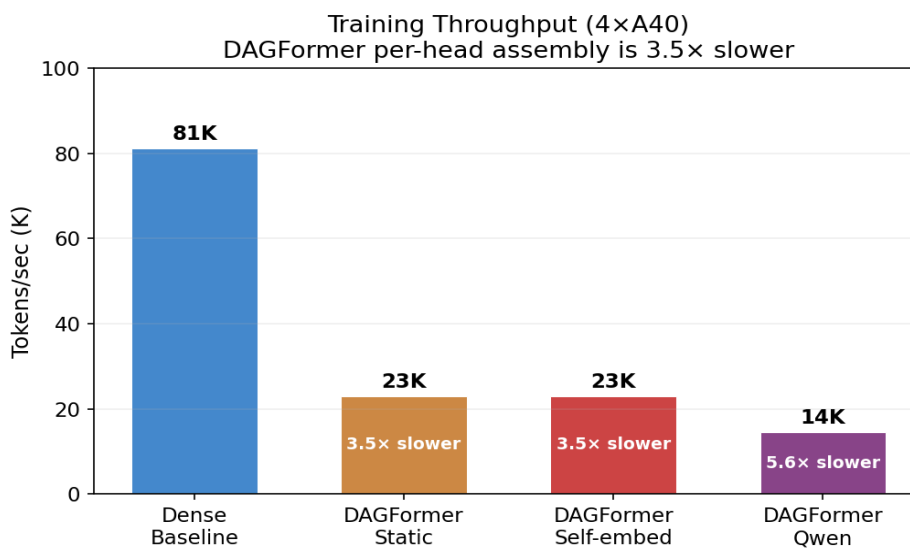


Figure 7. Training throughput. DAGFormer's per-head input assembly (einsum over A matrix) is 3.5x slower than standard batched attention. Qwen adds another 1.6x from the external encoder. This is the root cause of the NLL gap: same wall time, 3.5x fewer tokens seen.

Key Findings from Batch 1:

- **Self-embed learns sparse topology** (mean $A=0.42$). This is the first experiment where the predictor meaningfully deviates from dense.
- **Learned topology helps:** soft NLL (5.75) < own-baseline NLL (5.96), a 0.21 nat improvement on the same undertrained model.
- **Static predictor fails:** mean $A=0.98$, predictor gradient ~ 0 . Without input-dependent signal, the

predictor cannot learn.

- **3.5x throughput overhead is the bottleneck:** the model sees far fewer tokens than the baseline, so the base model is severely undertrained.

4. Current Experiments (In Progress)

To address the throughput bottleneck, we are testing five new strategies. All are currently submitted to the cluster.

4.1 Addressing Throughput: Staged & Alternating Training

Staged training

Load a fully-trained dense baseline checkpoint (10K steps), then switch to DAGFormer forward for an additional 2,000 steps. The base model starts with good language modeling ability; only needs to adapt to the new routing. Predictor starts fresh (from `init_logit=15` $\approx$ dense).

Alternating training

Train from scratch, but alternate between standard dense forward (fast, 9 steps) and DAGFormer forward (slow, 1 step) in a 9:1 ratio. Both step types update the base model; the predictor only updates on DAGFormer steps. Expected throughput: $\sim 90\%$ of baseline.

Self-embed without detach

Same as staged, but do not detach the embedding before feeding to the predictor. Gradients from the predictor's topology loss flow back into the embedding layer, encouraging the embedding to produce representations useful for topology prediction (not just language modeling).

4.2 Improving Predictor Input Quality

The original self-embed predictor applies **mean pooling** over raw token embeddings (before any transformer layers). This is essentially a bag-of-words — all positional and syntactic information is lost. We propose two improved designs:

Mini-encoder (independent)

A separate embedding table (`vocab_size` $\times$ 256) plus a 2-layer transformer encoder (`dim=256`, 4 heads). Learns its own contextual text representation optimized purely for topology prediction, completely independent of the LLM. $\sim 41\text{M}$ parameters (25.7M in embedding table).

Context-embed (shared embedding)

Reuses the LLM's embedding table (no extra vocab parameters), projects down ($1024 \rightarrow 256$), then applies a 2-layer transformer encoder. Much fewer parameters ($\sim 16\text{M}$) while still gaining contextual mixing. The LLM embedding is detached so the encoder's gradients don't interfere with LLM training.

#	Name	Strategy	Base Model	Predictor Input	Steps
1	Staged	Load baseline ckpt + DAGFormer	Pretrained (10K steps)	Self-embed (mean pool)	2,000
2	Alternating	9:1 standard: DAGFormer	From scratch	Self-embed (mean pool)	10,000
3	No-detach	Load baseline ckpt + no <code>.detach()</code>	Pretrained (10K steps)	Self-embed (unfrozen)	2,000
4	Mini-encoder	Load baseline ckpt	Pretrained (10K steps)	Independent 2L transformer	2,000
5	Context-embed	Load baseline ckpt	Pretrained (10K steps)	Shared embed + 2L transformer	2,000

Table 3. Five new experiments currently running. Key comparison axes: (1) staged vs alternating, (2) detach vs no-detach, (3) bag-of-words vs contextual predictor input, (4) independent vs shared embedding.

5. Discussion & Next Steps

5.1 The Core Tension

DAGFormer faces a fundamental speed-quality tradeoff: the per-head input assembly (computing $input_j = base + \sum A[i,j] \cdot head_output[i]$) requires iterating over layers sequentially and performing large einsum operations, making it $\sim 3.5\times$ slower than standard batched attention. More DAGFormer steps $\rightarrow$ better topology, but fewer total tokens $\rightarrow$ worse base model. The five current experiments explore different points on this tradeoff curve.

5.2 Open Questions for Discussion

- **Does context-dependent topology matter?** The static predictor failed, but it may simply need a different optimization approach (e.g., population-based). If a single fixed topology suffices, the predictor is unnecessary at inference time.
- **Can we amortize the overhead?** The $3.5\times$ slowdown comes from per-head routing. Possible mitigations: sparse A (skip zero gates), fused kernels, or only applying DAGFormer to a subset of layers.
- **What is the right predictor architecture?** Mean-pooled embeddings are too crude; a full external encoder (Qwen) is too expensive. The mini-encoder and context-embed designs are middle ground — but is 2 transformer layers enough?
- **Optimal model scale for topology learning?** 300M may be too small for topology to matter (fewer redundant heads). Would 1B or 3B show clearer benefits?
- **Distillation approach:** Train a dense teacher, then distill into a DAGFormer student with sparse topology. The student could match the teacher's performance with fewer active connections (efficiency gain at inference).

5.3 Proposed Future Recipes

Based on our findings, we propose several additional recipes for the team to discuss and prioritize:

Recipe	Description	Rationale
Aggressive alternating	50:1 or 100:1 standard:DAGFormer	Near-baseline throughput; predictor gets minimal but possibly sufficient signal
Layer-subset DAGFormer	Only apply A to last 4 layers	Reduce overhead to $\sim 1.5\times$; early layers may not need routing
Sparse A forward	Skip einsum for $A[i,j] < 0.01$	Exploit learned sparsity for speed; mean_A=0.42 means 58% can be skipped
Distillation	Dense teacher $\leftrightarrow$ sparse student	Decouple topology learning from language modeling; reduce inference cost
Curriculum	Short seq (256) first, then increase	Faster DAGFormer steps at small seq_len; gradually increase complexity

Recipe	Description	Rationale
Scale up	1B model, from scratch	More heads = more redundancy = more room for topology optimization

Table 4. Proposed future recipes for team discussion.

5.4 Summary

We have established that: (1) midtraining topology onto pretrained models does not work — the loss landscape is flat at $A=1$; (2) pretraining from scratch enables topology learning (self-embed achieves $\text{mean_A}=0.42$ with learned sparse routing that outperforms its own dense baseline); (3) the main bottleneck is now the 3.5x throughput overhead of DAGFormer's per-head forward pass. Our current experiments test five strategies to address this. The team can propose and prioritize additional recipes from the list above.